

CS-GY 6223

Operating Systems

Final Project

Team Members:

Name : Nigel Saurino

Net ID : ns5329

NYU ID : N11316907

Name : Sidharth Purohit

Net ID : sp6365

NYU ID : N10520397

Introduction

- We chose to implement our strace command natively in the shell by checking the text for the first command line argument against a pre-stored command string “strace”. We then parsed the character buffer from the shell into an array of arguments (options and a command). We set global variables in the kernel based on those options and executed the command from the shell. On command execution, we added functionality in syscall.c to print/store the appropriate system calls based on the options that we set in our global variables during parsing in the shell. We then built a ring buffer that stores the strings (character arrays) of the last N trace messages. We use a system call dump() to retrieve these strings to user space (the shell) and a loop to print them to the screen or to a file based on the user’s command. You can follow Nigel’s repository to check the entire end to end implementation.

Tasks

1. Get familiar with Linux strace

- Use strace in Anubis to learn about echo command or any other commands of your choice. Show a list of system call being made, total number of calls, time for running strace on particular command. (screenshots of your run should be enough for this task)
- Pick 4 random system call (ex: nmap, write, open, etc) of your choice and explain its functionality for one command that you run (ex: echo, ls, cd, etc) in 2-3 sentences.
 - Command: **cat README.md**
 - **System Call - Open:** The Open system call tries to open the filename that’s passed as a CLI argument. It returns the file descriptor number that the read, write and close commands will use for their operations. It takes a permission argument to determine if it is read/write/create enabled. It will return -1 if it fails to open the file.
 - **System Call - Read:** The Read system call takes the file descriptor that it is reading from (our open file), the location to store the read bytes and the total bytes it will read. It returns the number of bytes it has read or -1 if it failed to read the contents from the file descriptor.
 - **System Call - Write:** The Write system call takes the file descriptor it writes to, the contents it’s going to write and the total bytes it’s writing to the file descriptor all as arguments. Its return value is the total bytes it wrote to the file descriptor (the console in the case of cat) or -1 if the Write command fails.
 - **System Call - Close:** The Close system call receives the file descriptor of the file that we opened and closes it. It returns 0 if it succeeds and -1 if it fails.
 - Example below:

```

anubis@anubis-ide < master@4b83b2a > : ~/final-project-371525eb-sp6365
[0] % strace cat README.md
execve("/bin/cat", ["cat", "README.md"], 0x7ffeb0df6908 /* 46 vars */) = 0
brk(NULL) = 0x55acd32f7000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=34804, ...}) = 0
mmap(NULL, 34804, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fe2b89c4000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0@\n\2\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0755, st_size=1839792, ...}) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe2b89c2000
mmap(NULL, 1852680, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7fe2b87fd000
mprotect(0x7fe2b8822000, 1662976, PROT_NONE) = 0
mmap(0x7fe2b8822000, 1355776, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x25000) = 0x7fe2b8822000
mmap(0x7fe2b896d000, 303104, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x170000) = 0x7fe2b896d000
mmap(0x7fe2b89b8000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ba000) = 0x7fe2b89b8000
mmap(0x7fe2b89be000, 13576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7fe2b89be000
close(3) = 0
arch_prctl(ARCH_SET_FS, 0x7fe2b89c3580) = 0
mprotect(0x7fe2b89b8000, 12288, PROT_READ) = 0
mprotect(0x55acd2a73000, 4096, PROT_READ) = 0
mprotect(0x7fe2b89f7000, 4096, PROT_READ) = 0
munmap(0x7fe2b89c4000, 34804) = 0
brk(NULL) = 0x55acd32f7000
brk(0x55acd32f7000) = 0x55acd32f7000
openat(AT_FDCWD, "/usr/lib/locale/locale-archive", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=3041456, ...}) = 0
mmap(NULL, 3041456, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7fe2b8516000
close(3) = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x2), ...}) = 0
openat(AT_FDCWD, "README.md", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20, ...}) = 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7fe2b84f4000
read(3, "jashfohf0DFDHDl;GFHE", 131072) = 20
write(1, "jashfohf0DFDHDl;GFHE", 20) = 20
read(3, "", 131072) = 0
munmap(0x7fe2b84f4000, 139264) = 0
close(3) = 0
close(1) = 0
close(2) = 0
exit_group(0) = ?
+++ exited with 0 +++

```

2. Building strace in xv6

- Command: strace on

When typing "strace on" in the terminal, the mode of strace is on and therefore the next type in command will be traced. The system call list will be printed on screen in format pid (process id), command name, system call name, return value.

- **Strace on:** This command will stay persistent using a variable "tracing" flag that is set on and off in conjunction with a "traced" process specific flag that tags processes as traced or not traced.
- Example below is for **both** Strace on and Strace off:

```

init: starting sh
$ strace on
$ echo hi
TRACE: pid = 3 | command_name = echo | syscall = exec | return value: 0

hTRACE: pid = 3 | command_name = echo | syscall = write | return value: 1

iTRACE: pid = 3 | command_name = echo | syscall = write | return value: 1

TRACE: pid = 3 | command_name = echo | syscall = write | return value: 1

TRACE: pid = 3 | command_name = echo | syscall = exit | return value: 2

$ strace off
$ echo hi
hi
$ 

```

- Command: strace off

When typing "strace off" in the terminal, the mode of strace is off and therefore the next type in command won't be traced and nothing is print on screen besides the result of the commands.

- **Strace off:** Uses the same "tracing" flag and process-specific "traced" variables to flip the flags off and end the trace for all following commands.
- Example above in "Strace on" section of analysis.

- Command: strace run <command>

Instead of turning on and off strace, we create "strace run" to directly point tracing to the current process that executes the command. For example: when typing "strace run echo hello" in the terminal, we get the output tracing of echo hello.

- **Strace run <command>:** The Strace run <command> is part of the if-ladder command parser that we built directly into the shell (sh.c in xv6). Our implementation is designed to run strace for a command provided in the same line and will only trace that process before ending the trace for subsequent commands. It is not designed to be run with flags. The flag command format will be listed in the sections below. Similar to the strace on/off command, this functionality sets the "tracing" flag, parses out the command the user is targeting for the trace, runs the command and then sets the "tracing" flag off to end the trace.
- Example below:

```

$ strace run echo hello
TRACE: pid = 2 | command_name = echo | syscall = exec | return value: 0

hTRACE: pid = 2 | command_name = echo | syscall = write | return value: 1
eTRACE: pid = 2 | command_name = echo | syscall = write | return value: 1
lTRACE: pid = 2 | command_name = echo | syscall = write | return value: 1
lTRACE: pid = 2 | command_name = echo | syscall = write | return value: 1
oTRACE: pid = 2 | command_name = echo | syscall = write | return value: 1

TRACE: pid = 2 | command_name = echo | syscall = write | return value: 1
TRACE: pid = 2 | command_name = echo | syscall = exit | return value: 2

init: starting sh
$ echo hello
hello
$ 

```

- Command: strace dump

Implement a kernel memory that will save N number of latest events. This N number can be configurable by using define in XV6. In other words, it can be hard code but the way to implement is to use define to declare a variable called N with certain value. When "strace dump" command is called, print all events that saved in kernel memory.

- **Strace dump:** Our strace dump functionality records the past N trace messages as char* variables in a ring buffer of size N. Unfortunately, we had to hardcode N to be a specific size because we ran out of time figuring out how to implement an array of global character arrays for our sysbuf char* pointer to point to. This means that ultimately, our implementation is limited to an N of 10 because we had to create 10 separate char[1000] variables to store the last 10 trace messages instead of creating a dynamically sized array of char[1000] memory blocks. We believe we could have solved the dynamic array declaration/definition with syntax like `extern char trace_messages[1000][N];` but ran out of time to test and implement the change. Strace dump utilizes the index of the most recent system call (callcount) that was stored in our buffer to print the trace message log in the shell in order from the most recent system call to the oldest. We implemented a `dump(char*, int)` system call to retrieve the char* stored in the ring buffer at a specific index and handled the print loop in the shell.
- Example below:

```

$ strace -s -e write echo hello
hTRACE: pid = 5 | command_name = echo | syscall = write | return value: 1

eTRACE: pid = 5 | command_name = echo | syscall = write | return value: 1

lTRACE: pid = 5 | command_name = echo | syscall = write | return value: 1

lTRACE: pid = 5 | command_name = echo | syscall = write | return value: 1

oTRACE: pid = 5 | command_name = echo | syscall = write | return value: 1

TRACE: pid = 5 | command_name = echo | syscall = write | return value: 1

init: starting sh
$ strace -s cat README.md
TRACE: pid = 7 | command_name = cat | syscall = exec | return value: 0

TRACE: pid = 7 | command_name = cat | syscall = open | return value: 3

TRACE: pid = 7 | command_name = cat | syscall = read | return value: 11

Hello WorldTRACE: pid = 7 | command_name = cat | syscall = write | return value: 11

TRACE: pid = 7 | command_name = cat | syscall = read | return value: 0

TRACE: pid = 7 | command_name = cat | syscall = close | return value: 0

TRACE: pid = 7 | command_name = cat | syscall = exit | return value: 2

init: starting sh
$ strace -f -e open cat abc.txt
TRACE: pid = 8 | command_name = cat | syscall = open | return value: -1

cat: cannot open abc.txt
init: starting sh
$ strace dump
TRACE: pid = 8 | command_name = cat | syscall = open | return value: -1
TRACE: pid = 7 | command_name = cat | syscall = exit | return value: 2
TRACE: pid = 7 | command_name = cat | syscall = close | return value: 0
TRACE: pid = 7 | command_name = cat | syscall = read | return value: 0
TRACE: pid = 7 | command_name = cat | syscall = write | return value: 11
TRACE: pid = 7 | command_name = cat | syscall = read | return value: 11
TRACE: pid = 7 | command_name = cat | syscall = open | return value: 3
TRACE: pid = 7 | command_name = cat | syscall = exec | return value: 0
TRACE: pid = 5 | command_name = echo | syscall = write | return value: 1
TRACE: pid = 5 | command_name = echo | syscall = write | return value: 1
$ █

```

- Trace child process

Write a program that uses fork to spawn child processes. Utilize kernel memory that you have

previously implemented to store trace output of all processes. Perform strace on this program and print all trace of processes that you have spawn.

- For this program we are trying to perform a simple fork operation and capture the child and the parent processes for successful fork execution. We loop in 3 times to fork a parent process, just to see how the particular parent_id and child_id operate. If the fork is not performed and a -1 is returned we simply output that the fork() operation was not performed and execute it otherwise. We then try to utilize the kernel memory to store trace output of all the processes we executed. And then we simply perform the strace operation to verify and observe our result. So in the screenshots below we are performing two separate things as expected in the question prompt, the first one being a simple implementation of the **ForknStrace** program followed by tracing the execution of that program in the next screenshot.
- Fork and trace children example below:

```
init: starting sh
$ ForknStrace
Process is getting traced.
TRACE: pid = 3 | command_name = ForknStrace | syscall = fork | return value: 4
TRACE: pid = 3 | command_name = ForknStrace | syscall = wait | return value: 4
TRACE: pid = 3 | command_name = ForknStrace | syscall = fork | return value: 5
TRACE: pid = 3 | command_name = ForknStrace | syscall = wait | return value: 5
TRACE: pid = 3 | command_name = ForknStrace | syscall = fork | return value: 6
TRACE: pid = 3 | command_name = ForknStrace | syscall = wait | return value: 6
TRACE: pid = 3 | command_name = ForknStrace | syscall = strace | return value: 0

Both the Processes & forks are now traced.
TRACE: pid = 3 | command_name = ForknStrace | syscall = fork | return value: 7
TRACE: pid = 7 | command_name = ForknStrace | syscall = open | return value: -1
TRACE: pid = 7 | command_name = ForknStrace | syscall = close | return value: -1
TRACE: pid = 7 | command_name = ForknStrace | syscall = exit | return value: 2
TRACE: pid = 3 | command_name = ForknStrace | syscall = wait | return value: 7
TRACE: pid = 3 | command_name = ForknStrace | syscall = fork | return value: 8
TRACE: pid = 8 | command_name = ForknStrace | syscall = open | return value: -1
TRACE: pid = 8 | command_name = ForknStrace | syscall = close | return value: -1
TRACE: pid = 8 | command_name = ForknStrace | syscall = exit | return value: 2
TRACE: pid = 3 | command_name = ForknStrace | syscall = wait | return value: 8
TRACE: pid = 3 | command_name = ForknStrace | syscall = fork | return value: 9
TRACE: pid = 9 | command_name = ForknStrace | syscall = open | return value: -1
TRACE: pid = 9 | command_name = ForknStrace | syscall = close | return value: -1
TRACE: pid = 9 | command_name = ForknStrace | syscall = exit | return value: 2
TRACE: pid = 3 | command_name = ForknStrace | syscall = wait | return value: 9
TRACE: pid = 3 | command_name = ForknStrace | syscall = strace | return value: 0

Process not being traced.
$
```

- Trace output of the fork and trace children program below (command is cut-off but was strace on, then ForknStrace)::

```
TRACE: pid = 13 | command_name = ForknStrace | syscall = write | return value: 1
TRACE: pid = 13 | command_name = ForknStrace | syscall = strace | return value: 0
TRACE: pid = 13 | command_name = ForknStrace | syscall = fork | return value: 14
TRACE: pid = 13 | command_name = ForknStrace | syscall = wait | return value: 14
TRACE: pid = 13 | command_name = ForknStrace | syscall = fork | return value: 15
TRACE: pid = 13 | command_name = ForknStrace | syscall = wait | return value: 15
TRACE: pid = 13 | command_name = ForknStrace | syscall = fork | return value: 16
TRACE: pid = 13 | command_name = ForknStrace | syscall = wait | return value: 16
TRACE: pid = 13 | command_name = ForknStrace | syscall = strace | return value: 0

Both the Processes & forks are now traced.
TRACE: pid = 13 | command_name = ForknStrace | syscall = fork | return value: 17
TRACE: pid = 17 | command_name = ForknStrace | syscall = open | return value: -1
TRACE: pid = 17 | command_name = ForknStrace | syscall = close | return value: -1
TRACE: pid = 17 | command_name = ForknStrace | syscall = exit | return value: 2
TRACE: pid = 13 | command_name = ForknStrace | syscall = wait | return value: 17
TRACE: pid = 13 | command_name = ForknStrace | syscall = fork | return value: 18
TRACE: pid = 18 | command_name = ForknStrace | syscall = open | return value: -1
TRACE: pid = 18 | command_name = ForknStrace | syscall = close | return value: -1
TRACE: pid = 18 | command_name = ForknStrace | syscall = exit | return value: 2
TRACE: pid = 13 | command_name = ForknStrace | syscall = wait | return value: 18
TRACE: pid = 13 | command_name = ForknStrace | syscall = fork | return value: 19
TRACE: pid = 19 | command_name = ForknStrace | syscall = open | return value: -1
TRACE: pid = 19 | command_name = ForknStrace | syscall = close | return value: -1
TRACE: pid = 19 | command_name = ForknStrace | syscall = exit | return value: 2
TRACE: pid = 13 | command_name = ForknStrace | syscall = wait | return value: 19
TRACE: pid = 13 | command_name = ForknStrace | syscall = strace | return value: 0

Process not being traced.
$ █
```


3. Building option for strace

- **NOTE ON IMPLEMENTATION:** For all of our flags, we implemented them to have the command run in the same line as the `strace <flag>` call as a demonstration that it only runs once like the “run” implementation above. We thought that this would be less confusing for the user who may not expect the flags to terminate after one run when they try to turn it on in one command and then use the flag for subsequent commands that they want to trace. Our implementation for the `strace` command with flags takes the form: `strace <flag> <flag e if included> <system call if flag e set> <command>` (i.e. `strace -s -e write echo hi`). We used similar logic to capture all of the flags for each shell command. We looped through every argument in the shell command and checked if any of them matched the four options that we were asked to implement. We used global kernel variables to store the flags and a system call to set them after they’d been recorded in a local variable. We do all of the command/option parsing and execution of `strace` natively in the shell.
- Option: `-e <system call name>` ...When option flag `-e` is provided followed by a system call name (ex: `write`), we will print only that system call. If no such system call is made in the command, print nothing.

Notes: Notice the first `strace` run is similar to regular “`echo hello`” run above in figure 1. Next command is “`strace -e write`” which activates tracking `write` system calls. We run “`echo hello`” again, observing that only the `write` system call is printed. One more run “`echo hello`”, everything is back to normal without tracing only the `write` system call. Similar ideas should be done in option `-s` and `-f`.

- **Option -e:** Our `-e` flag captures only the system call that is specified by the user through the shell. It prints the system calls that have matching system call ID’s and stores them in our ring buffer.
- Example below:

```
$ strace -e write echo hi
hTRACE: pid = 9 | command_name = echo | syscall = write | return value: 1
iTRACE: pid = 9 | command_name = echo | syscall = write | return value: 1

TRACE: pid = 9 | command_name = echo | syscall = write | return value: 1
init: starting sh
$
```

- Option: `-s`...When option flag `-s` is provided, print only successful system call.
 - **Option -s:** Our `-s` flag captures only the successful system calls from the shell command, prints them and stores them to our ring buffer. We check that the return value stored in our process “`eax`” pointer is not `-1` to confirm the syscall succeeded on execution. By

comparing this output and the output of the -f flag screenshot, you can see that each has omitted the other's system calls. (Failed Open() call is omitted from the output below).

- Example below:

```
init: starting sh
$ strace -s cat abc.txt
TRACE: pid = 10 | command_name = cat | syscall = exec | return value: 0

cTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
aTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
tTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
:TRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
TRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
cTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
aTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
nTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
nTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
oTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
tTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
TRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
oTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
pTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
eTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
nTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
TRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
aTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
bTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
cTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
.TRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
tTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
xTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
tTRACE: pid = 10 | command_name = cat | syscall = write | return value: 1

TRACE: pid = 10 | command_name = cat | syscall = write | return value: 1
TRACE: pid = 10 | command_name = cat | syscall = exit | return value: 2
init: starting sh
$
```

- Option: -f...When option flag -f is provided, print only failed system call.

- **Option -f:** Our -f flag captures only the failed system calls from the shell command, prints them and stores them to our ring buffer. We check that the return value stored in our process "eax" pointer is -1 to confirm the syscall failed on execution. By comparing this output and the output of the -s flag screenshot, you can see that each has omitted the other's system calls (Only the failed Open() is output below).

```
init: starting sh
$ strace -f cat abc.txt
TRACE: pid = 11 | command_name = cat | syscall = open | return value: -1

cat: cannot open abc.txt
init: starting sh
$ █
```

- Option runs only once

When an option is called (ex: "strace -e write" follow by "echo hello"), the result of strace with only the write system call is printed only once for the following command "echo hello". If typing the next follow command for example "ls", the option -e is turned off and all system calls will be printed. In other words, options are used once per command.

- **Option runs only once:** We implemented the flags to run like "strace run" to avoid any confusion for the user. So the flags and the trace are only executed for the current command before being cleared for the next shell command. Below, we execute just the write system calls before turning the generic trace command on and executing it for all system calls on the same "echo hello" command.
- Example below:

```

init: starting sh
$ strace -e write echo hello
hTRACE: pid = 12 | command_name = echo | syscall = write | return value: 1

eTRACE: pid = 12 | command_name = echo | syscall = write | return value: 1

lTRACE: pid = 12 | command_name = echo | syscall = write | return value: 1

lTRACE: pid = 12 | command_name = echo | syscall = write | return value: 1

oTRACE: pid = 12 | command_name = echo | syscall = write | return value: 1

TRACE: pid = 12 | command_name = echo | syscall = write | return value: 1

init: starting sh
$ echo hello
hello
$ strace on
$ echo hello
TRACE: pid = 15 | command_name = echo | syscall = exec | return value: 0

hTRACE: pid = 15 | command_name = echo | syscall = write | return value: 1

eTRACE: pid = 15 | command_name = echo | syscall = write | return value: 1

lTRACE: pid = 15 | command_name = echo | syscall = write | return value: 1

lTRACE: pid = 15 | command_name = echo | syscall = write | return value: 1

oTRACE: pid = 15 | command_name = echo | syscall = write | return value: 1

TRACE: pid = 15 | command_name = echo | syscall = write | return value: 1

TRACE: pid = 15 | command_name = echo | syscall = exit | return value: 2

$ 

```

- Extra credits: Combine options Implement these 2 commands:
 - "strace -s -e <system call name>": print only successful system call name.
 - **strace -s -e <system call name>**: We implemented the parser to check the -s/-f flags first and then check for -e flag and the matching system calls/return values. In the example below, we run cat which fails on the open() system call but succeeds for exec(). We ask it to catch both system calls only if they succeed and notice that the exec() prints while the open() does not, verifying our implementation. Return values act as additional confirmation.
 - Example below:

```

$ strace -s -e exec cat abc.txt
TRACE: pid = 13 | command_name = cat | syscall = exec | return value: 0

cat: cannot open abc.txt
init: starting sh
$ strace -s -e open cat abc.txt
cat: cannot open abc.txt
init: starting sh
$ 

```

– “strace -f -e <system call name>”: print only failed system call name.

- **strace -f -e <system call name>**: We implemented the parser to check the -s/-f flags first and then check for -e flag and the matching system calls/return values. We use the same example as above but with the -f flag instead of the -s flag and notice that the opposite system calls print from the -s flag output above. Return values act as additional confirmation.
- Example below:

```
init: starting sh
$ strace -f -e exec cat abc.txt
cat: cannot open abc.txt
init: starting sh
$ strace -f -e open cat abc.xt
TRACE: pid = 18 | command_name = cat | syscall = open | return value: -1
cat: cannot open abc.xt
init: starting sh
$
```

4. Write output of strace to file

- Find an implementation of choice to write strace output to file.
- For example: by providing option: -o <filename> or editing content of README.
 - **strace -o <filename>**: The -o flag uses the standard open(), read(), write() system calls in order to try to open a file with the specified name. If the file does not exist, we create one and overwrite the file contents with the new trace message log from our ring buffer using the dump() system call. We spent many hours trying to figure out why our file didn't show up in the Anubis file directory and finally realized that the file was being created in the xv6 directory instead. This is demonstrated by creating the file, calling the “ls” command on the xv6 directory and finally calling the cat command on the file that we created (and is seen in the “ls” output). In the example below, we try to open the file and catch the failed open system call. Then we create and write to the same file that we tried to open but didn't exist (trace_log.txt). Finally, we cat the contents of the file to the screen and “ls” to demonstrate that it has been created in xv6's directory.
 - Example below:

```

init: starting sh
$ strace -f -e open cat abc.txt
TRACE: pid = 4 | command_name = cat | syscall = open | return value: -1

cat: cannot open abc.txt
init: starting sh
$ strace -o trace_log.txt
init: starting sh
$ cat trace_log.txt
TRACE: pid = 4 | command_name = cat | syscall = open | return value: -1
TRACE: pid = 3 | command_name = cat | syscall = exit | return value: 2
TRACE: pid = 3 | command_name = cat | syscall = close | return value: 0
TRACE: pid = 3 | command_name = cat | syscall = read | return value: 0
TRACE: pid = 3 | command_name = cat | syscall = write | return value: 11
TRACE: pid = 3 | command_name = cat | syscall = read | return value: 11
TRACE: pid = 3 | command_name = cat | syscall = open | return value: 3
TRACE: pid = 3 | command_name = cat | syscall = exec | return value: 0
TRACE: pid = 2 | command_name = echo | syscall = exit | return value: 2
TRACE: pid = 2 | command_name = echo | syscall = write | return value: 1
$ ls
.          1 1 512
..         1 1 512
README.md 2 2 11
cat        2 3 15688
echo       2 4 14792
forktest   2 5 9404
grep       2 6 17800
init       2 7 15380
kill       2 8 14900
ln         2 9 14796
ls         2 10 17272
mkdir      2 11 14944
rm         2 12 14932
sh         2 13 38116
stressfs   2 14 15496
usertests  2 15 65360
wc         2 16 16412
zombie     2 17 14468
Forkn5trace 2 18 15492
w         2 19 14072
race       2 20 15232
console    3 21 0
-o         2 22 10020
trace_log.txt 2 23 10020
$ 

```

5. Application of strace

Write a small program that produces unexpected behavior such as race condition, delay output, crash on condition, memory leak or your choice of implementation. Run strace on this program.

- Explain your program and its unexpected behavior.
 - In this program we are trying to capture erratic behavior while performing the memory leak and arithmetically invalid operation. Here we are allocating memory dynamically to an integer pointer block of size 5. And we simply add numeric values to our dynamic array, next we perform a divide by 0 operation on the entire block, i.e. every array object. Now we know if we do not perform the memory free operation on this dynamically allocated block after our usage is over, it will lead to a memory leak error; also divide-by-0 action is an invalid operation. This is shown in the screenshot below. The following

abruption is captured by our trace program and is shown in the screenshot after this question. Below is the implementation of our file which will produce this race condition.

```
user > race.c > ...
1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  // Here we are simply creating a memory block and performing a divide by 0 operation on it
6  //program fails for 2 cases -
7  // 1. if memory is not freed
8  // 2. if divide by 0 is performed
9  void simple_leak(){
10     int n=5;
11     int m=0;
12     int *ptr = (int*)malloc(n * sizeof(int));
13     for (int i = 0; i < n; ++i) {
14         ptr[i] = i * 100;
15         ptr[i]/=m; //fail here as divide by 0
16     }
17     //free(ptr); Fails here as well, as we are not freeing the pointer we allocated.
18     //exit();
19 }
20 int main(int argc, char *argv[]) {
21     simple_leak();
22     return 0;
23 }
```

- (xv6) Does your strace implementation provide anything useful that can tell you something about this unexpected behavior?
 - Yes, our strace program shows the following abruption by first allocating the space and showing us all the system calls involved while doing this task. Post allocation, we tried printing all of the values present in our array. So the **sbrk** system call allocated the memory for us. The program crashes because we haven't performed the memory deallocation process and are also dividing by 0 in our program. This behavior is captured in our implementation.

```
init: starting sh
$ strace on
$ race
TRACE: pid = 3 | command_name = race | syscall = exec | return value: 0

TRACE: pid = 3 | command_name = race | syscall = sbrk | return value: 12288

pid 3 race: trap 0 err 0 on cpu 0 eip 0x6a addr 0x0--kill proc
$ □
```

- (Linux) Implement the same program on Linux in Anubis (there is slight variation of C language between Linux and XV6), does Linux strace provide more information and is it better to help you debug this issue?
 - The Linux based implementation of **strace** did help us in determining how to debug the program when memory free is not being performed and when the divide by 0 operation on any numerical(integer in our case) is done. It helped us in determining what all system calls we performed in the execution, including all the writes, memory allocation etc. operations correctly. We believe our implementation of strace on xv6 is equally good in determining where exactly the program fails, along with the error. - **trap 0 err 0 on cpu 0 eip 0x6a addr 0x0 -kill proc**, which helped us in determining the exact memory leak and divide by 0 error occurring in our program.

```

raceAn.c > simple_leak
1
2  #include<stdlib.h>
3  #include<stdio.h>
4
5
6
7  // Here we are simply creating a memory block and performing a divide by 0 operation on it
8  //program fails for 2 cases -
9  // 1. if memory is not freed
10 // 2. if divide by 0 is performed
11
12 void simple_leak(){
13     int n=5;
14     int m=0;
15
16     int *ptr = (int*)malloc(n * sizeof(int));
17     for (int i = 0; i < n; ++i) {
18         ptr[i] = i *100;
19         ptr[i]/=m; //fail here as divide by 0
20     }
21     free(ptr);
22 }
23
24 int main(int argc, char *argv[]) {
25     simple_leak();
26     return (0);
27 }
28
29

```



```

anubis@anubis-ide < master@dca46ee > : ~/final-project-371525eb-sp6365 [
[0] % gcc raceAn.c -o rc
anubis@anubis-ide < master@dca46ee > : ~/final-project-371525eb-sp6365 [
[0] % strace ./rc
execve("/.rc", ["/.rc"], 0x7ffed474f300 /* 46 vars */) = 0
brk(NULL) = 0x56483db8e000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=34804, ...}) = 0
mmap(NULL, 34804, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f57c2cfd000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0\0\1\0\0\0\2\0\0\0\0\0"... , 832) = 832
fstat(3, {st_mode=S_IFREG|0755, st_size=1839792, ...}) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f57c2cfb000
mmap(NULL, 1852680, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f57c2b36000
mprotect(0x7f57c2b5b000, 1662976, PROT_NONE) = 0
mmap(0x7f57c2b5b000, 1355776, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x25000) = 0x7f57c2b5b000
mmap(0x7f57c2ca6000, 303104, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x170000) = 0x7f57c2ca6000
mmap(0x7f57c2cf1000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ba000) = 0x7f57c2cf1000
mmap(0x7f57c2cf7000, 13576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f57c2cf7000
close(3) = 0
arch_prctl(ARCH_SET_FS, 0x7f57c2cfc540) = 0
mprotect(0x7f57c2cf1000, 12288, PROT_READ) = 0
mprotect(0x56483c73a000, 4096, PROT_READ) = 0
mprotect(0x7f57c2d30000, 4096, PROT_READ) = 0
munmap(0x7f57c2cfd000, 34804) = 0
brk(NULL) = 0x56483db8e000
brk(0x56483dbaf000) = 0x56483dbaf000
--- SIGFPE {si_signo=SIGFPE, si_code=FPE_INTDIV, si_addr=0x56483c7381c1} ---
+++ killed by SIGFPE (core dumped) +++
[1] 12782 floating point exception strace ./rc
anubis@anubis-ide < master@dca46ee > : ~/final-project-371525eb-sp6365 [
[136] %

```